

Summary — SynapseOS: Designing a Conversational Interface Layer for Personal Computing

Willard C. Soriano

Department of Information Technology

Mapua University – Makati

wcsoriano@mymail.mapua.edu.ph

Scope and Selection Criteria

This review surveys prior work on natural language and LLM-mediated interfaces to computing systems, with the goal of positioning SynapseOS — a conversational interface layer intended to replace conventional graphical desktop interaction — within the current research landscape. Eight studies were selected from peer-reviewed venues and arXiv preprints in computer science (subcategories cs.OS, cs.AI, cs.HC, and cs.CL), published between 2024 and 2026. Inclusion required that the work either (a) implement a working natural language interface to system-level or cross-application functionality, (b) propose an architectural framework for such interfaces, or (c) provide an evaluation benchmark for them. Industry blog posts, GitHub-only artifacts without accompanying papers, and domain-specific assistants (e.g., medical or database chatbots) were excluded.

Comparative Matrix of Related Works

The matrix below compares the eight studies along seven dimensions chosen to reflect the engineering and evaluation concerns most relevant to SynapseOS: how much of the user's computing experience the system covers, the modality through which the user provides input, the mechanism by which natural language is translated into system action, the language engine employed, the type and scale of evaluation conducted, the operating system targeted, and the principal limitation reported by the authors themselves.

Table 1: Comparative analysis of related works on conversational and LLM-mediated interfaces for personal computing, organized by coverage scope, input modality, action execution mechanism, language engine, evaluation, target operating system, and reported limitations.

No	Ref	System	Coverage Scope	Input Modality	Action Mechanism	Execution LLM/Engine	Evaluation	Target OS	Reported Limitation
1	[1]	NL2SH	Shell (CLI)	Text	Command translation (NL to Bash)	Fine-tuned (multiple)	LLMs	Manually verified test set of 600 instruction-command pairs and training set of 40,939 pairs, with novel functional equivalence heuristic reaching 95% confidence	Translation accuracy still limited by ambiguity and lack of system context
2	[2]	NaSh	Shell (CLI)	Text	NL shell with guardrails layer	LLM (model-agnostic)		Design paper with worked examples; classified under cs.OS	LLM outputs may be unintended or unexplainable,

No	Ref	System	Coverage Scope	Input Modality	Action Mechanism	Execution LLM/Engine	Evaluation	Target OS	Reported Limitation
3	[3]	Multimodal GUI Architecture	Single application	Voice GUI	API binding through Context over MVVM	Model LLM with real-time speech recognition	Architectural prototype, no large user study	OS-agnostic (application layer)	requiring guardrails for user error recovery Most production applications were not designed with speech in mind
4	[4]	UFO ²	Cross-application desktop	Text	Hybrid: native APIs combined with vision-based parsing of UI Automation tree	Multimodal LLM	System-level deployment with task benchmarks across Windows applications	Windows	Earlier desktop agents suffered from shallow OS integration and fragile screenshot-only interaction
5	[5]	ScreenAgent	Cross-application desktop	Text	GUI simulation via mouse and keyboard inferred from screenshots	Vision-Language Model trained on ScreenAgent Dataset	Model Performance comparable to GPT-4V with stronger positioning	OS-agnostic	Vision-only grounding is less reliable than approaches with API access
6	[6]	WinClick	Cross-application desktop	Text	Visual grounding screenshots LLM-based alignment	GUI from with Multimodal LLM with grounding pre-training	WinSpot benchmark for Windows GUI grounding	Windows	Most existing GUI agents rely on structured data formats like DOM or HTML, which are inaccessible across general desktop environments
7	[7]	OSWorld	Benchmark environment	Text (agent input)	N/A (evaluation harness)	Tested across GPT, 369 real-world computer tasks; Claude, Gemini, Qwen, humans achieve 72.36% success Ubuntu, Mixtral, CogAgent, UI- while best model reaches 12.24%, Windows, TARS, Operator, Agent- with weaknesses in GUI grounding macOS S and operational knowledge			Reveals that no current agent approaches human-level system competence
8	[8]	LLM-Brained GUI Agents Survey	Survey systems across	N/A	N/A	N/A (synthesis)	Literature review identifying core components and gaps	All	Open challenges include accurate element localization, knowledge

No	Ref	System	Coverage Scope	Input Modality	Action Mechanism	Execution LLM/Engine	Evaluation	Target OS	Reported Limitation
									retrieval, long-horizon planning, and safety-aware execution control

Analysis of Findings

Three patterns emerge clearly from the matrix.

The first pattern is a sharp bifurcation in coverage scope. The literature concentrates at two poles: shell-level translation and per-application command interpretation on one end (rows 1, 2, 3), and cross-application desktop automation at the other (rows 4, 5, 6). No reviewed system implements a working full-OS conversational replacement that has been empirically evaluated with users. The closest analogue, UFO², is explicitly framed as a "Desktop AgentOS" but operates as an automation layer atop the conventional Windows GUI rather than as a replacement for it.

The second pattern concerns the action execution mechanism. The cross-application cluster splits between vision-only approaches (ScreenAgent, WinClick) and hybrid approaches that combine native APIs with vision (UFO²). UFO²'s authors explicitly characterize the vision-only path as fragile, while WinClick's authors note that the API-style alternatives depend on structured data such as DOM trees that are unavailable on general desktops. This tension is unresolved in the literature and represents a concrete design decision SynapseOS must confront.

The third pattern concerns evaluation. With the exception of OSWorld, which provides task-level execution benchmarks but not user studies, the surveyed systems either report no formal user evaluation or rely on small architectural prototypes. The OSWorld results themselves are sobering: leading models complete only roughly one in eight real computer tasks that humans complete at over seventy percent. This suggests that any claim of usability for a conversational OS must be supported by direct empirical comparison against the conventional GUI workflow it intends to replace, not by demonstration alone.

Identified Research Gaps

Synthesizing across the matrix, four research gaps emerge that SynapseOS is positioned to address.

The first is the absence of full-OS conversational systems implemented and evaluated on Linux. NaSh addresses Linux but only at the shell level; UFO², ScreenAgent, and WinClick are Windows-centric or OS-agnostic at the application layer. A Linux-based, OS-wide conversational layer remains unbuilt in the academic literature.

The second is the lack of empirical comparison against conventional desktop interaction. None of the implemented systems reviewed report a controlled user study comparing task completion time, error rate, or satisfaction between the conversational interface and the equivalent GUI workflow. SynapseOS's stated goal of evaluating viability and efficiency directly addresses this gap.

The third is the unresolved architectural question of how natural language intent should be translated into system action when both API access and visual grounding are partial. SynapseOS's design will need to take a position on this tradeoff, which prior work has identified but not settled.

The fourth concerns user heterogeneity. The reviewed evaluations, where they exist, treat users as a homogeneous group. SynapseOS's ambition of accommodating "all types of users" is unrealistic at thesis scale, but a comparative evaluation across two or three user populations (for example, novices versus power users, or users with motor accessibility needs versus general users) would represent a contribution that no reviewed work has attempted.

Implications for SynapseOS

The literature establishes that LLM-mediated computing interfaces are technically feasible at narrow scopes (shell, single application) and increasingly viable at the desktop automation layer, but that no working full-OS conversational replacement has been built or empirically validated, particularly on Linux. SynapseOS's contribution can therefore be framed not as a feasibility demonstration alone, which prior systems already establish, but as the first empirically evaluated, Linux-based, OS-wide conversational interface, with an explicit comparison against the conventional desktop workflow it proposes to replace. The matrix above provides both the methodological precedent for this evaluation, in OSWorld, and the architectural precedents, in UFO² and NaSh, that SynapseOS can build upon.

References

- [1] Westenfelder, F. J., Ferraro, M. A., Hossain, M., Reddy, M., Adiga, A., Vullikanti, A., Marathe, M., & Vempala, S. (2025). LLM-Supported Natural Language to Bash Translation. arXiv preprint arXiv:2502.06858. <https://arxiv.org/pdf/2502.06858>
- [2] Gyawali, B. R., Achalla, S., Kallas, K., & Kumar, S. (2025). NaSh: Guardrails for an LLM-Powered Natural Language Shell. arXiv preprint arXiv:2506.13028. <https://arxiv.org/abs/2506.13028>
- [3] van Dam, H. G. W. (2025). A Multimodal GUI Architecture for Interfacing with LLM-Based Conversational Assistants. arXiv preprint arXiv:2510.06223. <https://arxiv.org/pdf/2510.06223>
- [4] Zhang, C., He, S., Qian, J., Li, B., Li, L., Qin, S., Kang, Y., Ma, M., Liu, G., Lin, Q., Rajmohan, S., Zhang, D., & Zhang, Q. (2025). UFO²: The Desktop AgentOS. arXiv preprint arXiv:2504.14603. <https://arxiv.org/pdf/2504.14603>
- [5] Niu, R., Li, J., Wang, S., Fu, Y., Hu, X., Leng, X., Kong, H., Chang, Y., & Wang, Q. (2024). ScreenAgent: A Vision Language Model-driven Computer Control Agent. arXiv preprint arXiv:2402.07945. <https://arxiv.org/pdf/2402.07945>
- [6] Hui, T., Zhang, Y., Jiang, B., Sun, J., Cheng, F., Lin, X., & Yang, Y. (2025). WinClick: GUI Grounding with Multimodal Large Language Models. arXiv preprint arXiv:2503.04730. <https://arxiv.org/pdf/2503.04730>
- [7] Xie, T., Zhang, D., Chen, J., Li, X., Zhao, S., Cao, R., Hua, T. J., Cheng, Z., Shin, D., Lei, F., Liu, Y., Xu, Y., Zhou, S., Savarese, S., Xiong, C., Zhong, V., & Yu, T. (2024). OSWorld: Benchmarking Multimodal Agents for Open-Ended Tasks in Real Computer Environments. Advances in Neural Information Processing Systems 37 (NeurIPS 2024). arXiv:2404.07972. <https://arxiv.org/abs/2404.07972>
- [8] Zhang, C., He, S., Qian, J., Li, B., Li, L., Qin, S., Kang, Y., Ma, M., Liu, G., Lin, Q., Rajmohan, S., Zhang, D., & Zhang, Q. (2024). Large Language Model-Brained GUI Agents: A Survey. arXiv preprint arXiv:2411.18279. <https://arxiv.org/abs/2411.18279>